

# ML / AI Engineer — Complete Agent Prompt

Lagos Emergency Intelligence Network · NacosCuab Hackathon '26

Everything your AI coding agent needs to execute your role — start to finish.

## How to Use This Document

This document IS the prompt for your AI coding agent (Claude Code, Cursor, Copilot Workspace, or any agentic coding tool). Each section maps to a task block. Feed the agent one task block at a time, in order. Every block contains: the exact goal, the file path to create or edit, the complete code to produce, and a verification gate before moving on. Do not skip gates — a broken gate means the next block will fail.

|                |                                                                          |
|----------------|--------------------------------------------------------------------------|
| Your role      | ML/AI Engineer — you own the entire AI pipeline and your share of setup  |
| Your repo path | All AI code lives under ai/ in the project root                          |
| Your branch    | feature/ai-pipeline — branch off main after Phase 0                      |
| Merge policy   | PR to main after each Phase gate passes. Do not commit directly to main. |
| Commit message | feat(ai): — use this prefix on every commit                              |
| Agent tool     | Claude Code recommended. Feed one TASK block per session.                |

## PHASE 0 — SETUP & ENVIRONMENT — ML/AI SLICE | Hour 0 – 2

**Context for agent:** You have just accepted a GitHub organisation invitation for NacosCuabHackathon26. The repo already exists. Your job in this phase is to clone it, create your working branch, set up the Python AI environment, create the `ai/` directory structure, write the requirements file, and make a skeleton commit so the repo shows activity from Hour 0.

### TASK 0.1 — CLONE & BRANCH

```
# Run these commands in your terminal after accepting the GitHub invite

git clone https://github.com/NacosCuabHackathon26/.git

cd

git checkout -b feature/ai-pipeline

git push -u origin feature/ai-pipeline
```

### TASK 0.2 — CREATE AI DIRECTORY STRUCTURE

Tell your agent to create this exact folder layout:

```
ai/
ai/__init__.py
ai/classifier.py # Component 1 – NLP Classifier
ai/severity.py # Component 2 – Severity Scorer
ai/router.py # Component 3 – Route Optimiser
ai/forecaster.py # Component 4 – Incident Forecaster
ai/pipeline.py # Orchestrator – wires all 4 together
ai/models/ # Saved .joblib model files go here
ai/data/ # Synthetic training data CSVs go here

scripts/
```

```
scripts/train_severity.py # Standalone training script
scripts/train_forecaster.py
scripts/generate_data.py # Synthetic data generator
scripts/test_pipeline.py # Manual end-to-end test
```

#### TASK 0.3 — REQUIREMENTS-AI.TXT

Create **requirements-ai.txt** in the project root with exactly:

```
scikit-learn==1.4.2
joblib==1.4.2
numpy==1.26.4
pandas==2.2.2
anthropic==0.28.0 # For NLP classifier – Option A
python-dotenv==1.0.1
statsmodels==0.14.2 # For ARIMA forecaster
httpx==0.27.0 # Async HTTP for classifier API calls
```

```
pip install -r requirements-ai.txt
```

#### TASK 0.4 — .ENV SETUP

Create **.env** in project root (already in .gitignore — confirm this):

```
ANTHROPIC_API_KEY=sk-ant-YOUR_KEY_HERE
AI_MODE=api # options: api | offline
CLASSIFIER_MODEL=claude-haiku-20240307
```

**CRITICAL:** Confirm **.env** is in **.gitignore** before the first commit. Exposing an API key to a public repo during a hackathon is a disqualification risk. Run: `grep '.env' .gitignore` — it must appear.

#### TASK 0.5 — SKELETON COMMIT

```
# Add placeholder content to each ai/*.py file so they are importable
# Each file should contain at minimum:
# """LEIN AI Pipeline — """

git add ai/ scripts/ requirements-ai.txt
git commit -m "feat(ai): scaffold AI pipeline directory structure"
git push
```

|            |          |                                                                                            |
|------------|----------|--------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 0.A | python -c 'import sklearn, joblib, anthropic, pandas, numpy, statsmodels'<br>— zero errors |
| ■ CRITICAL | Gate 0.B | ai/ directory exists with all 6 .py files importable                                       |
| ■ CRITICAL | Gate 0.C | .env exists locally, confirmed absent from git tracked files                               |
| ■ CRITICAL | Gate 0.D | feature/ai-pipeline branch pushed and visible on GitHub                                    |

## PHASE 2 — AI PIPELINE BUILD | Hour 8 – 20

**Context for agent:** The backend engineer has finished Phase 1. All 8 API endpoints exist and return mock JSON. Your job now is to build the real AI behind them — four components in sequence. After each component you will slot it into the matching FastAPI endpoint by editing the backend engineer's stub. Coordinate with them on the exact import path they expect.

## PHASE 2 — COMPONENT 1 — NLP EMERGENCY CLASSIFIER | Hour 8 – 12

**Goal:** Given a free-text emergency description (possibly in English or Pidgin), return a structured classification: emergency type, confidence score, and extracted keywords.

[FULL FILE: AI/CLASSIFIER.PY](#)

```
"""
LEIN AI Pipeline — Component 1: NLP Emergency Classifier
Supports two modes via AI_MODE env var:
api — calls Claude claude-haiku via Anthropic SDK (fast, accurate)
offline — TF-IDF + LogisticRegression trained on synthetic data
"""

import os, json, re
from dotenv import load_dotenv
load_dotenv()

EMERGENCY_TYPES = ['medical', 'fire', 'security', 'accident', 'other']

HIGH_RISK_KEYWORDS = [
    'unconscious', 'not breathing', 'bleeding', 'stabbed', 'shot',
    'gun', 'fire', 'burning', 'trapped', 'collapsed', 'seizure',
```





```

return _offline_model

def _train_offline_model():
    from sklearn.pipeline import Pipeline
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.linear_model import LogisticRegression
    import joblib, os

    EXAMPLES = [
        ('person collapsed not breathing', 'medical'),
        ('chest pain heart attack', 'medical'),
        ('unconscious on the floor', 'medical'),
        ('seizure convulsing', 'medical'),
        ('bleeding heavily from head', 'medical'),
        ('building on fire flames spreading', 'fire'),
        ('gas explosion fire caught', 'fire'),
        ('kitchen fire burning', 'fire'),
        ('armed robbery in progress', 'security'),
        ('gunshots heard multiple', 'security'),
        ('stabbing attack knife', 'security'),
        ('car crash multiple injured', 'accident'),
        ('road accident bus overturned', 'accident'),
        ('motorcycle collision injured', 'accident'),
        ('noise disturbance neighbour', 'other'),
        ('lost child missing person', 'other'),
    ]

    texts, labels = zip(*EXAMPLES)

```





```

else:
    result = classify_offline(normalized)
    except Exception as e:
        print(f'Classifier fallback triggered: {e}')
        result = classify_offline(normalized)
    result['keywords'] = keywords
    result['raw_input'] = description
    return result

```

|            |          |                                                                                                                            |
|------------|----------|----------------------------------------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 1.A | python -c "from ai.classifier import classify; print(classify('person not breathing'))" — returns dict with type='medical' |
| ■ CRITICAL | Gate 1.B | classify('fire don catch for building') — Pidgin test, returns type='fire'                                                 |
| ■ CRITICAL | Gate 1.C | classify('noise from neighbour') — returns type='other', confidence < 0.7                                                  |
| ■ TODO     | Commit   | git commit -m 'feat(ai): NLP emergency classifier — API + offline modes'                                                   |

## PHASE 2 — COMPONENT 2 — SEVERITY SCORER | Hour 12 – 15

**Goal:** Predict a priority score (1–10) for each incident using structured features. This score drives the dispatch queue — highest score appears first on the dispatcher dashboard.

[FULL FILE: SCRIPTS/GENERATE\\_SEVERITY\\_DATA.PY — RUN THIS FIRST TO GENERATE TRAINING DATA](#)

```
import pandas as pd, numpy as np, os

np.random.seed(42)

N = 500

TYPE_BASE = {'medical':8.5, 'fire':8.0, 'accident':7.5, 'security':6.5, 'other':3.0}

types = np.random.choice(list(TYPE_BASE.keys()), N,
p=[0.35,0.20,0.20,0.15,0.10])

hours = np.random.randint(0, 24, N)

days = np.random.randint(0, 7, N)

LGA_WEIGHTS = {'ikeja':0.9, 'surulere':0.85, 'islandLG':0.95, 'alimosho':0.7,
'eti_osa':0.8, 'kosofe':0.75, 'mushin':0.88, 'oshodi':0.82}

lgas = np.random.choice(list(LGA_WEIGHTS.keys()), N)

lga_w = np.array([LGA_WEIGHTS[l] for l in lgas])

hints = np.random.randint(1, 6, N)

kw_flags = np.random.randint(0, 5, N)

peak = ((hours>=6)&(hours<=9)) | ((hours>=16)&(hours<=20))

base_scores = np.array([TYPE_BASE[t] for t in types])

scores = (base_scores * 0.45 + lga_w * 1.5 + hints * 0.3 +
kw_flags * 0.4 + peak.astype(float) * 0.8 +
np.random.normal(0, 0.4, N))

scores = np.clip(scores, 1, 10).round(2)
```

```

type_enc = {'medical':1.0, 'fire':0.9, 'accident':0.8, 'security':0.7, 'other':0.3}
df = pd.DataFrame({
    'type_weight': [type_enc[t] for t in types],
    'hour_of_day': hours, 'day_of_week': days, 'lga_risk': lga_w,
    'severity_hint': hints, 'keyword_flags': kw_flags,
    'is_peak_hour': peak.astype(int), 'score': scores
})
os.makedirs('ai/data', exist_ok=True)
df.to_csv('ai/data/severity_training.csv', index=False)
print(f'Generated {N} training examples. Score range: {scores.min():.1f}-{scores.max():.1f}')

```

**FULL FILE: [SCRIPTS/TRAIN\\_SEVERITY.PY](#)**

```

import pandas as pd, joblib, os

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('ai/data/severity_training.csv')
FEATURES = ['type_weight', 'hour_of_day', 'day_of_week', 'lga_risk',
    'severity_hint', 'keyword_flags', 'is_peak_hour']
X, y = df[FEATURES], df['score']
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_tr, y_tr)
mae = mean_absolute_error(y_te, model.predict(X_te))
print(f'Severity model MAE: {mae:.3f} (target < 0.8)')
os.makedirs('ai/models', exist_ok=True)

```

```
joblib.dump({'model': model, 'features': FEATURES}, 'ai/models/severity.joblib')
print('Model saved to ai/models/severity.joblib')
```

**FULL FILE: AI/SEVERITY.PY**

```
"""LEIN AI Pipeline – Component 2: Severity Scorer"""
import os, joblib, numpy as np

TYPE_WEIGHTS = {'medical':1.0, 'fire':0.9, 'accident':0.8, 'security':0.7, 'other':0.3}
LGA_WEIGHTS = {
    'ikeja':0.9, 'surulere':0.85, 'island':0.95, 'alimosho':0.7,
    'eti_osa':0.8, 'kosofe':0.75, 'mushin':0.88, 'oshodi':0.82,
    'default':0.75
}

_model_cache = None

def _load():
    global _model_cache
    if _model_cache: return _model_cache
    path = os.path.join(os.path.dirname(__file__), 'models', 'severity.joblib')
    _model_cache = joblib.load(path)
    return _model_cache

def score(incident_type: str, hour: int, day: int, lga: str,
          severity_hint: int, keyword_flags: int) -> float:
    """Returns priority score 1-10. Higher = more urgent."""
    saved = _load()
    model, features = saved['model'], saved['features']
    is_peak = 1 if (6<=hour<=9) or (16<=hour<=20) else 0
```

```

lga_risk = LGA_WEIGHTS.get(lga.lower(), LGA_WEIGHTS['default'])
type_w = TYPE_WEIGHTS.get(incident_type.lower(), 0.3)
X = [[type_w, hour, day, lga_risk, severity_hint, keyword_flags, is_peak]]
raw = model.predict(X)[0]
# Safety fallback: high keyword count always scores high
if keyword_flags >= 3: raw = max(raw, 8.0)
return round(float(np.clip(raw, 1.0, 10.0)), 2)

```

**Training sequence for agent:** (1) python scripts/generate\_severity\_data.py (2) python scripts/train\_severity.py (3) Confirm MAE printed is below 0.8 (4) Confirm ai/models/severity.joblib exists

|            |          |                                                                                                           |
|------------|----------|-----------------------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 2.A | python scripts/generate_severity_data.py — prints '500 training examples'                                 |
| ■ CRITICAL | Gate 2.B | python scripts/train_severity.py — prints MAE < 0.8                                                       |
| ■ CRITICAL | Gate 2.C | python -c "from ai.severity import score; print(score('medical',8,1,'ikeja',5,4))" — returns value > 8.0  |
| ■ CRITICAL | Gate 2.D | python -c "from ai.severity import score; print(score('other',14,3,'default',1,0))" — returns value < 5.0 |
| ■ TODO     | Commit   | git commit -m 'feat(ai): severity scorer — RandomForest trained on 500 synthetic incidents'               |

## PHASE 2 — COMPONENT 3 — ROUTE OPTIMISER | Hour 15 – 18

**Goal:** Given an incident location (lat/lng) and a list of available responders, select the optimal responder and return estimated ETA in minutes. Uses Haversine distance + Lagos traffic weights.

[FULL FILE: AI/ROUTER.PY](#)

```
"""LEIN AI Pipeline – Component 3: Route Optimiser"""

import math

from datetime import datetime

# Lagos LGA traffic multipliers by hour bucket
# 1.0 = free flow, 2.5 = heavy congestion
TRAFFIC_PROFILE = {
    'ikeja': [1.2,1.1,1.0,1.0,1.1,1.4,2.0,2.5,2.3,1.8,1.6,1.5,
1.6,1.5,1.5,1.6,2.0,2.5,2.4,2.0,1.7,1.5,1.3,1.2],
    'surulere': [1.1,1.0,1.0,1.0,1.1,1.3,1.8,2.3,2.1,1.7,1.5,1.4,
1.5,1.4,1.4,1.5,1.9,2.3,2.2,1.9,1.6,1.4,1.2,1.1],
    'island': [1.3,1.2,1.1,1.1,1.2,1.5,2.2,2.8,2.5,2.0,1.8,1.7,
1.8,1.7,1.7,1.8,2.2,2.8,2.6,2.2,1.9,1.6,1.4,1.3],
    'default': [1.2,1.1,1.0,1.0,1.1,1.4,1.9,2.3,2.1,1.7,1.5,1.4,
1.5,1.4,1.4,1.5,1.9,2.3,2.1,1.8,1.5,1.3,1.2,1.1],
}

# Average speed in Lagos km/h (adjusted by traffic)
BASE_SPEED_KMH = 40.0

def haversine(lat1, lng1, lat2, lng2) -> float:
    """Returns distance in km between two coordinates."""
```

```

R = 6371.0

dlat = math.radians(lat2 - lat1)
dlng = math.radians(lng2 - lng1)
a = (math.sin(dlat/2)**2 +
math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) *
math.sin(dlng/2)**2)
return R * 2 * math.asin(math.sqrt(a))

def get_traffic_multiplier(lga: str, hour: int) -> float:
    profile = TRAFFIC_PROFILE.get(lga.lower(), TRAFFIC_PROFILE['default'])
    return profile[hour % 24]

def optimise(incident_lat: float, incident_lng: float,
responders: list[dict],
hour: int = None) -> dict:
    """
    responders: list of dicts with keys:
    id, lat, lng, lga, capacity_weight (0-1, higher=better)
    Returns: {responder_id, distance_km, eta_minutes, score}
    """
    if hour is None:
        hour = datetime.now().hour
    available = [r for r in responders if r.get('available', True)]
    if not available:
        return {'error': 'no_available_responders', 'responder_id': None}
    scored = []
    for r in available:

```



```

dist = haversine(incident_lat, incident_lng, r['lat'], r['lng'])
traffic = get_traffic_multiplier(r.get('lga','default'), hour)
capacity = r.get('capacity_weight', 0.8)
# Lower score = better (cost function)
cost = (dist * traffic) / capacity
speed = BASE_SPEED_KMH / traffic
eta = round((dist / speed) * 60, 1) # minutes
scored.append({'id': r['id'], 'dist': dist, 'eta': eta,
'cost': cost, 'traffic': traffic})
best = min(scored, key=lambda x: x['cost'])
return {
'responder_id': best['id'],
'distance_km': round(best['dist'], 2),
'eta_minutes': best['eta'],
'traffic_factor':best['traffic'],
'score': round(best['cost'], 4)
}

```

|            |          |                                                                                                                                                                                    |
|------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 3.A | from ai.router import optimise; result = optimise(6.52, 3.38, [{'id':'R1','lat': 6.53,'lng':3.37,'lga':'ikeja','capacity_weight':0.9,'available':True}]) — returns eta_minutes > 0 |
| ■ CRITICAL | Gate 3.B | Test with all responders available=False — returns error: no_available_responders                                                                                                  |
| ■ CRITICAL | Gate 3.C | Test two responders: one 0.5km away, one 5km away — picks the closer one                                                                                                           |

■ **TODO**

**Commit**

git commit -m 'feat(ai): route optimiser — Haversine + Lagos traffic weights'

## PHASE 2 — COMPONENT 4 — INCIDENT FORECASTER | Hour 18 – 20

**Goal:** Predict incident counts for the next 6 hours by type. Powers the analytics dashboard forecast table. Uses RandomForest with lag features trained on 90 days of synthetic data with realistic Lagos patterns.

[FULL FILE: SCRIPTS/GENERATE\\_FORECAST\\_DATA.PY](#)

```
import pandas as pd, numpy as np, os

np.random.seed(0)

hours = pd.date_range('2026-01-01', periods=90*24, freq='h')
rows = []

for dt in hours:
    h, d = dt.hour, dt.dayofweek

    is_peak = 1 if (6<=h<=9) or (16<=h<=20) else 0
    is_weekend = 1 if d >= 5 else 0

    base = 3 + is_peak*6 + is_weekend*2

    rows.append({
        'hour': h, 'day_of_week': d, 'is_peak': is_peak,
        'is_weekend': is_weekend,
        'medical': max(0, int(np.random.poisson(base*0.4))),
        'fire': max(0, int(np.random.poisson(base*0.2))),
        'security': max(0, int(np.random.poisson(base*0.2))),
        'accident': max(0, int(np.random.poisson(base*0.15))),
        'other': max(0, int(np.random.poisson(base*0.05))),
    })

df = pd.DataFrame(rows)

os.makedirs('ai/data', exist_ok=True)
```

```
df.to_csv('ai/data/forecast_training.csv', index=False)
print(f'Generated {len(df)} hourly records')
```

**FULL FILE: SCRIPTS/TRAIN\_FORECASTER.PY**

```
import pandas as pd, joblib, os
from sklearn.ensemble import RandomForestRegressor
TYPES = ['medical', 'fire', 'security', 'accident', 'other']
df = pd.read_csv('ai/data/forecast_training.csv')
FEATURES = ['hour', 'day_of_week', 'is_peak', 'is_weekend']
models = {}
for t in TYPES:
    m = RandomForestRegressor(n_estimators=50, random_state=42)
    m.fit(df[FEATURES], df[t])
    models[t] = m
    print(f'{t}: trained')
os.makedirs('ai/models', exist_ok=True)
joblib.dump({'models': models, 'features': FEATURES},
            'ai/models/forecaster.joblib')
print('Forecaster saved to ai/models/forecaster.joblib')
```

**FULL FILE: AI/FORECASTER.PY**

```
"""LEIN AI Pipeline – Component 4: Incident Forecaster"""
import os, joblib
from datetime import datetime, timedelta

_cache = None

def _load():
```

```

global _cache

if _cache: return _cache

path = os.path.join(os.path.dirname(__file__), 'models', 'forecaster.joblib')
_cache = joblib.load(path)
return _cache

def forecast(hours_ahead: int = 6) -> list[dict]:
    """Returns predicted incident counts for next N hours."""
    saved = _load()
    models, features = saved['models'], saved['features']
    now = datetime.now()
    results = []
    for i in range(1, hours_ahead + 1):
        dt = now + timedelta(hours=i)
        h, d = dt.hour, dt.weekday()
        is_peak = 1 if (6<=h<=9) or (16<=h<=20) else 0
        is_weekend = 1 if d >= 5 else 0
        X = [[h, d, is_peak, is_weekend]]
        preds = {t: max(0, round(float(models[t].predict(X)[0]),1))
        for t in models}
        preds['total'] = round(sum(preds.values()), 1)
        preds['hour'] = dt.strftime('%H:00')
        preds['is_peak'] = bool(is_peak)
        results.append(preds)
    return results

```

■ CRITICAL

Gate 4.A

python scripts/generate\_forecast\_data.py — prints '2160 hourly records'

|            |          |                                                                                                                                         |
|------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 4.B | python scripts/train_forecaster.py — prints trained for all 5 types                                                                     |
| ■ CRITICAL | Gate 4.C | from ai.forecaster import forecast; r = forecast(6) — returns list of 6 dicts each with medical/fire/security/accident/other/total keys |
| ■ TODO     | Commit   | git commit -m 'feat(ai): incident forecaster — per-type RandomForest on 90-day synthetic data'                                          |

## PHASE 2 — PIPELINE ORCHESTRATOR — WIRE ALL 4 COMPONENTS | Hour 20 – 22

**Goal:** Create a single entry point that the FastAPI backend calls. The backend engineer only needs to call one function: `process_incident()`. This file is your handoff to them.

[FULL FILE: AI/PIPELINE.PY](#)

```
"""
LEIN AI Pipeline — Orchestrator
Single entry point for the FastAPI backend.
Backend engineer: import process_incident and route_incident from here.
"""

from datetime import datetime
from ai.classifier import classify
from ai.severity import score
from ai.router import optimise
from ai.forecaster import forecast

def process_incident(description: str, lat: float, lng: float,
lga: str, severity_hint: int) -> dict:
    """
    Full AI processing for a new incident report.
    Calls classifier then severity scorer.
    Returns enriched incident dict ready for DB insert.
    """

    now = datetime.now()

    # Step 1: Classify
    classification = classify(description)
```

```

# Step 2: Score severity
kw_count = len(classification.get('keywords', []))
priority = score(
    incident_type = classification['type'],
    hour = now.hour,
    day = now.weekday(),
    lga = lga,
    severity_hint = severity_hint,
    keyword_flags = kw_count
)
return {
    'ai_type': classification['type'],
    'ai_confidence': classification['confidence'],
    'ai_keywords': classification['keywords'],
    'ai_priority': priority,
    'ai_reasoning': classification.get('reasoning', ''),
}

def route_incident(incident_lat: float, incident_lng: float,
    responders: list[dict]) -> dict:
    """Wrapper around route optimiser. Backend calls this for /assign."""
    return optimise(incident_lat, incident_lng, responders)

def get_forecast(hours_ahead: int = 6) -> list[dict]:
    """Wrapper for forecaster. Backend calls this for /forecast."""
    return forecast(hours_ahead)

```

**BACKEND INTEGRATION INSTRUCTIONS — HAND THIS TO YOUR BACKEND ENGINEER**



**In POST /classify endpoint:** Replace mock with: `from ai.pipeline import process_incident ai_result = process_incident(incident.description, incident.lat, incident.lng, incident.lga, incident.severity_hint)` Update incident record with `ai_result` fields. **In POST /assign endpoint:** Replace mock ETA with: `from ai.pipeline import route_incident route = route_incident(incident.lat, incident.lng, available_responders)` **In GET /forecast endpoint:** Replace stub with: `from ai.pipeline import get_forecast return get_forecast(hours_ahead=6)`

|            |          |                                                                                                                                                                                                               |
|------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate P.A | python scripts/test_pipeline.py (create this: calls <code>process_incident</code> with a sample description) — returns dict with <code>ai_type</code> , <code>ai_priority</code> , <code>ai_confidence</code> |
| ■ CRITICAL | Gate P.B | <code>ai_priority &gt; 8</code> when description contains 'unconscious not breathing' — life threat scores high                                                                                               |
| ■ CRITICAL | Gate P.C | <code>ai_priority &lt; 5</code> when description is 'noise from neighbours at night'                                                                                                                          |
| ■ TODO     | Commit   | <code>git commit -m 'feat(ai): pipeline orchestrator — wires all 4 AI components'</code>                                                                                                                      |
| ■ TODO     | PR       | Open pull request: <code>feature/ai-pipeline -&gt; main</code> . Title: 'feat: complete AI pipeline — 4 components'                                                                                           |

## PHASE 4 — INTEGRATION TESTING — ML/AI SLICE | Hour 28 – 36

**Goal:** Verify the full AI pipeline works end-to-end through the live API, not just in isolation. Run these tests against the running FastAPI server.

**FULL FILE: [SCRIPTS/TEST\\_PIPELINE.PY](#) — RUN THIS AGAINST THE LIVE SERVER**

```
import httpx, json

BASE = 'http://localhost:8000'

TEST_CASES = [
    # (description, expected_type, min_priority)
    ('Person unconscious not breathing on the floor', 'medical', 8.0),
    ('Building on fire flames spreading to next house', 'fire', 7.5),
    ('Armed robbery in progress at the shop', 'security', 6.5),
    ('Car accident multiple people injured on expressway', 'accident', 7.0),
    ('Noise disturbance from party next door', 'other', 1.0),
    ('Im don fall e no dey breathe', 'medical', 7.5), # Pidgin
    ('Fire don catch for building', 'fire', 7.0), # Pidgin
    ('Chest pain heart attack symptoms', 'medical', 8.5),
    ('Gunshots heard multiple times', 'security', 7.0),
    ('Child missing since morning', 'other', 2.0),
]

passed = failed = 0

for desc, exp_type, min_pri in TEST_CASES:
    # Step 1: report
    r1 = httpx.post(f'{BASE}/report', json={
        'description': desc, 'lat': 6.52, 'lng': 3.38,
```

```

'severity_hint': 3, 'lga': 'ikeja'
})

inc_id = r1.json()['incident_id']
# Step 2: classify
r2 = httpx.post(f'{BASE}/classify', json={'incident_id': inc_id})
result = r2.json()
ai_type = result.get('ai_type', '')
ai_pri = result.get('ai_priority', 0)
ok_type = ai_type == exp_type
ok_pri = ai_pri >= min_pri
status = 'PASS' if (ok_type and ok_pri) else 'FAIL'
if status == 'PASS': passed += 1
else: failed += 1

print(f'{status} | {desc[:45]:<45} | got={ai_type}({ai_pri}) exp={exp_type}({min_pri})')
print(f'\nResult: {passed}/10 passed, {failed} failed')

assert failed == 0, f'PIPELINE TEST FAILED: {failed} cases broken'

```

|            |          |                                                                                        |
|------------|----------|----------------------------------------------------------------------------------------|
| ■ CRITICAL | Gate 4.A | python scripts/test_pipeline.py — prints 10/10 passed                                  |
| ■ CRITICAL | Gate 4.B | GET /forecast returns list of 6 dicts with all type keys present                       |
| ■ CRITICAL | Gate 4.C | POST /classify response time under 500ms (measure with httpx elapsed)                  |
| ■ CRITICAL | Gate 4.D | Offline mode test: set AI_MODE=offline in .env, re-run test suite — still passes 8+/10 |
| ■ TODO     | Commit   | git commit -m 'test(ai): 10-case integration test suite — all passing'                 |

## PHASE 5 — DOCUMENTATION — AI SECTION OF README | Hour 40 – 44

**Goal:** Write the 'How AI Powers LEIN' section of the README. This is what judges read to score AI Integration (20 pts). Paste this block verbatim into README.md under the AI section heading.

### README CONTENT — COPY EXACTLY INTO README.MD

```
## How AI Powers LEIN

LEIN integrates four distinct AI components that work in sequence
for every emergency reported:

### 1. NLP Emergency Classifier (`ai/classifier.py`)
- What: Classifies free-text descriptions into emergency types
(medical, fire, security, accident, other)
- How: Zero-shot classification via Claude Haiku API with structured
prompting. Offline fallback: TF-IDF + Logistic Regression.
- Lagos-specific: Pidgin normalisation layer handles common Lagos
emergency phrases before classification.
- Output: `{type, confidence, keywords, raw_input}`

### 2. Severity Scorer (`ai/severity.py`)
- What: Predicts priority score 1-10 to rank the dispatch queue.
- How: RandomForestRegressor trained on 500 synthetic incidents.
Features: incident type, hour of day, LGA risk weight, citizen hint,
keyword flags, peak-hour indicator.
- Output: Float 1.0-10.0 (higher = more urgent)

### 3. Route Optimiser (`ai/router.py`)
- What: Selects optimal responder and estimates ETA.
```

```

- How: Haversine distance + per-LGA 24-hour traffic profiles.
Cost function: distance * traffic_weight / capacity_weight.
- Output: `{responder_id, distance_km, eta_minutes, traffic_factor}`

### 4. Incident Forecaster (`ai/forecaster.py`)
- What: Predicts incident counts for next 6 hours by type.
- How: Five RandomForest models (one per type) trained on 90 days
of synthetic hourly data with Lagos rush-hour and weekend patterns.
- Output: List of 6 hourly dicts with per-type predicted counts.

### AI Pipeline Entry Point
All four components are orchestrated through `ai/pipeline.py`.
The FastAPI backend calls three functions:
- `process_incident()` – classifier + severity scorer
- `route_incident()` – route optimiser
- `get_forecast()` – incident forecaster

```

## PHASE 5 — DEMO DAY — YOUR 60-SECOND SCRIPT | June 9, 2026

**This is the most important thing you rehearse.** The demo is the product. Practice this script until you can do it in under 60 seconds without looking at notes.

|                       |                                                                                                                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Second 0 – 10</b>  | Open the Citizen Interface on your phone browser. Say: 'A 45-year-old man has collapsed and is not breathing in Ikeja.' Type it into the report field. Hit Submit.                                                             |
| <b>Second 10 – 20</b> | Show the AI classification result appearing: type=MEDICAL, confidence=0.97, keywords=['unconscious','not breathing'], priority=9.4/10. Say: 'LEIN's NLP classifier identifies this instantly as a critical medical emergency.' |

|                       |                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Second 20 – 35</b> | Switch to the Dispatch Dashboard. Show the incident at the TOP of the queue in red. Say: 'The severity scorer ranked this 9.4 — it's first in the queue ahead of everything else.'                                                                        |
| <b>Second 35 – 48</b> | Click the incident. Show the map highlighting the nearest hospital (Lagos Island General) and the closest available ambulance unit 1.2km away with ETA 4 minutes. Say: 'The route optimiser factors in current Ikeja traffic and picks the fastest unit.' |
| <b>Second 48 – 58</b> | Click the Analytics tab. Point to the forecast table showing a spike predicted in 2 hours. Say: 'The forecaster predicted this rush-hour spike — dispatchers can pre-position units.'                                                                     |
| <b>Second 58 – 60</b> | Say: 'LEIN — built for Lagos, scalable to every Nigerian city. Four AI systems. 48 hours. One life saved.'                                                                                                                                                |

## Master Checklist — ML/AI Engineer

Run through this top-to-bottom before submission window opens at 11:59 PM June 7.

|        |                  |                                                                                 |
|--------|------------------|---------------------------------------------------------------------------------|
| ■ TODO | Env              | requirements-ai.txt installs clean. .env present locally. .env absent from git. |
| ■ TODO | Branch           | feature/ai-pipeline merged to main via PR. No uncommitted changes.              |
| ■ TODO | Classifier       | classify('person not breathing') returns type='medical', confidence > 0.85      |
| ■ TODO | Pidgin           | classify('fire don catch') returns type='fire'                                  |
| ■ TODO | Offline fallback | AI_MODE=offline still classifies correctly for all 5 types                      |
| ■ TODO | Severity model   | ai/models/severity.joblib exists. MAE < 0.8.                                    |
| ■ TODO | High priority    | score('medical', 8, 1, 'ikeja', 5, 4) returns > 8.5                             |
| ■ TODO | Low priority     | score('other', 14, 2, 'default', 1, 0) returns < 5.0                            |
| ■ TODO | Router           | optimise() picks closer responder when two options given                        |
| ■ TODO | No responders    | optimise() returns error dict gracefully when all busy                          |
| ■ TODO | Forecaster       | forecast(6) returns list of 6 dicts with all 5 type keys                        |
| ■ TODO | Forecaster model | ai/models/forecaster.joblib exists                                              |
| ■ TODO | Pipeline         | process_incident() returns dict with all 5 ai_ keys                             |
| ■ TODO | Integration      | python scripts/test_pipeline.py against live server — 10/10 pass                |

|                   |                         |                                                                         |
|-------------------|-------------------------|-------------------------------------------------------------------------|
| ■ <b>TODO</b>     | <b>Speed</b>            | POST /classify responds in under 500ms                                  |
| ■ <b>TODO</b>     | <b>API wired</b>        | Backend /classify calls process_incident(), not stub                    |
| ■ <b>TODO</b>     | <b>Forecast wired</b>   | Backend /forecast calls get_forecast(), not stub                        |
| ■ <b>TODO</b>     | <b>Assign wired</b>     | Backend /assign calls route_incident(), not stub                        |
| ■ <b>TODO</b>     | <b>README</b>           | 'How AI Powers LEIN' section in README.md — all 4 components documented |
| ■ <b>CRITICAL</b> | <b>Demo cached</b>      | Last 10 classification results cached. Demo works if API is down.       |
| ■ <b>CRITICAL</b> | <b>Demo rehearsed</b>   | 60-second script practiced at least twice end-to-end                    |
| ■ <b>CRITICAL</b> | <b>Submission alarm</b> | 11:45 PM alarm set for June 7. Repo URL saved. Form link bookmarked.    |

**You own the thing that wins this hackathon. Four AI components. Offline fallback. Integration tests. Clean documentation. Ship all of it and the team wins.**

**Built for Lagos. Scalable to every Nigerian city.**